

Real-Time Chat Application

20 Comprehensive Interview Questions & Detailed Answers (Technical & Behavioral)

Category 1: Technical Architecture & System Design

Q1: Why did you choose WebSockets over HTTP Long Polling or Server-Sent Events (SSE) for this real-time chat application?

Answer: HTTP is stateless and follows a client-initiated Request-Response pattern. For a chat app, HTTP Long Polling incurs heavy overhead because every request re-sends HTTP headers and opens/closes TCP connections repeatedly, wasting server CPU and network bandwidth. Server-Sent Events (SSE) are unidirectional (server-to-client only), requiring separate HTTP POST requests for sending messages. WebSockets establish a single, persistent, full-duplex TCP connection via an initial HTTP Upgrade handshake. This allows both client and server to exchange low-latency binary or text frames asynchronously with minimal overhead (~2 bytes frame header vs ~800 bytes HTTP header).

Q2: Standard browser WebSocket APIs do not allow custom HTTP headers (e.g. Authorization: Bearer <token>) during handshake. How did you secure WebSocket connections with JWT?

Answer: To bypass the browser restriction on custom headers during WebSocket instantiation, I passed the JWT as a URL query parameter (`ws://domain/ws?token=JWT_TOKEN`). During the WebSocket connection lifecycle in FastAPI, before calling `websocket.accept()`, the server extracts the query token, verifies its signature using HS256 and the `SECRET_KEY`, and extracts the user ID. If the token is invalid or expired, the server immediately rejects the handshake by raising an HTTP 401/1008 policy violation error. Once verified, the connection is bound to that user ID in the active connections mapping.

Q3: How did you manage database sessions within long-lived WebSocket connections to prevent connection pool exhaustion?

Answer: Injecting a database session via a standard FastAPI dependency (like `get_db`) into a long-lived WebSocket loop keeps the DB connection checked out for as long as the user stays connected (potentially hours), quickly exhausting the SQLAlchemy connection pool. To prevent this, I decoupled the socket lifetime from DB sessions. Inside the `ConnectionManager`, DB operations (saving messages, updating online status) use short-lived context managers (with `SessionLocal()` as `db`). The DB session is opened strictly for the database query/commit and closed immediately after, releasing the connection back to the pool.

Q4: If you scale this application horizontally across multiple backend server instances behind a load balancer, what breaks and how would you fix it?

Answer: Currently, active WebSocket connections are stored in an in-memory Python dictionary (`self.active_connections`). If User A is connected to Instance 1 and User B is connected to Instance 2, Instance 1 cannot route messages to User B because it lacks B's socket instance. To solve this horizontal scaling challenge, we integrate Redis Pub/Sub as a centralized message broker. When a message is sent to a room, the server instance publishes it to a Redis channel. All backend instances subscribe to Redis channels and forward incoming published messages to their locally connected WebSockets. This decouples socket state from message delivery.

Q5: Explain your database schema design for handling both public group chat rooms and private direct messages (DMs) in a single Message table.

Answer: The Message schema includes: id (Primary Key), sender_id (Foreign Key to User), room_id (Foreign Key to Room, Nullable), recipient_id (Foreign Key to User, Nullable), content (Text), created_at (DateTime), and is_read (Boolean). If room_id is populated and recipient_id is null, it is a public room message accessible by all members of that room. If recipient_id is populated and room_id is null, it is a private direct message. To query DM history between User A and User B, we filter for (sender=A AND recipient=B) OR (sender=B AND recipient=A), ordered by created_at DESC with limit/offset pagination.

Q6: How does FastAPI's asynchronous event loop (asyncio) handle thousands of concurrent WebSocket connections efficiently?

Answer: FastAPI is built on Starlette and ASGI (Asynchronous Server Gateway Interface), utilizing Python's asyncio event loop. Traditional WSGI servers (like Flask/Django) use thread-per-request models, where thousands of idle connections consume gigabytes of RAM in thread stacks. FastAPI's async def websocket_endpoint yields control back to the single-threaded event loop whenever waiting for network I/O (e.g., await websocket.receive_text()). The event loop handles thousands of concurrent open socket descriptors using OS-level I/O multiplexing primitives (epoll on Linux, kqueue on macOS, IOCP on Windows) with extremely low memory footprint.

Q7: How did you structure Pydantic schemas for data validation, and how did you resolve dynamic attributes like sender_username from ORM models?

Answer: I separated Pydantic schemas into UserBase/UserCreate/UserResponse, RoomCreate/RoomResponse, and MessageBase/MessageResponse to enforce strict request validation and response payload filtering. To supply sender_username in MessageResponse without redundant DB joins during serialization, I added a @property sender_username on the SQLAlchemy Message ORM model that returns self.sender.username if populated. With model_config = ConfigDict(from_attributes=True), Pydantic automatically evaluates ORM properties during model instantiation.

Q8: Explain the query strategy and logic behind paginating message history for chat rooms.

Answer: To retrieve chat history efficiently, the REST endpoint takes query parameters limit (default 50) and offset (default 0). The database query sorts by created_at DESC to fetch the most recent N messages first:
`db.query(Message).filter(Message.room_id == room_id).order_by(Message.created_at.desc()).limit(limit).offset(offset).all()`
Before returning the payload to the frontend, the list is reversed in memory (`list(reversed(messages))`) so the frontend receives them in ascending chronological order (oldest to newest), allowing smooth top-to-bottom rendering in the UI.

Q9: What is CORS, why did you add CORSMiddleware in FastAPI, and what production security adjustments are necessary?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP/WebSocket requests initiated from a frontend origin different from the backend server. In development, I added CORSMiddleware with `allow_origins=[*]` to allow the vanilla JS client running on various ports to interact with the API. In production, `allow_origins` must be restricted to specific trusted domain names (e.g., `https://chat.yourdomain.com`), `allow_credentials` set to True, and wildcard origins disabled to prevent unauthorized cross-origin requests.

Q10: How are real-time typing indicators implemented over WebSockets, and how did you prevent network spam through frontend debouncing?

Answer: When a user types in the input field, the frontend triggers a socket event { type: 'typing', payload: { is_typing: true, room_id: 1 } }. To avoid spamming the WebSocket on every keystroke, the frontend uses debouncing: a JavaScript timer (setTimeout) resets on each keypress. If no new key is pressed for 2 seconds, an event with is_typing: false is emitted. On the backend, the ConnectionManager broadcasts the typing status only to other active users in that room (excluding the sender). The recipient UI displays an animated typing bubble that automatically disappears upon receiving is_typing: false or timing out.

Q11: Explain your Docker multi-container configuration and why a health check is essential for MySQL in docker-compose.yml.

Answer: The project is containerized using a Dockerfile (Python 3.10-slim image with uvicorn entrypoint) and orchestrated via docker-compose.yml with three services: web (FastAPI), db (MySQL 8.0), and redis (Redis cache). When containers start concurrently, the MySQL container initializes its process before its database engine is ready to accept socket connections. Without a health check, FastAPI attempts to connect immediately and crashes. I defined a healthcheck in MySQL using mysqladmin ping, and configured web with depends_on: db: condition: service_healthy, guaranteeing FastAPI starts only after MySQL is fully ready.

Q12: How did you implement automated integration tests for your FastAPI endpoints without polluting the development/production database?

Answer: I wrote an automated test suite in tests/test_chat.py using pytest and FastAPI's TestClient. To isolate testing data, I used FastAPI's app.dependency_overrides[get_db] to swap the default database engine with a separate SQLite database file (test_chat.db). A pytest module fixture (setup_and_teardown_database) executes Base.metadata.create_all(bind=engine) before running tests (testing user registration, login, JWT protection, room creation) and automatically drops tables and deletes test_chat.db after execution.

Category 2: Behavioral & Scenario-Based Questions (STAR Method)

Q13: Describe a challenging technical bug you encountered during development and how you diagnosed and resolved it.

Answer: Situation: During initial WebSocket load testing, the server froze and stopped responding to new connections after a few minutes.

Task: Diagnose why WebSocket connections were hanging and blocking the server.

Action: I inspected server logs and thread states. I discovered that database sessions were being injected into long-lived WebSocket handler loops. Because the WebSocket connection remained open indefinitely, the SQLAlchemy session was never closed, holding onto database connection pool slots until all available connections were exhausted.

Result: I refactored the ConnectionManager to use short-lived context managers (with SessionLocal() as db:) scoped strictly to individual read/write operations. This freed connections immediately after commits, dropping active pool utilization to minimal levels and handling concurrent connections smoothly.

Category 1: Behavioral & Scenario-Based Questions

Q14: What key architectural trade-offs did you evaluate when choosing SQLite for development vs MySQL for production?

Answer: Situation: I needed a database strategy that allowed zero-configuration local development while being production-ready.

Task: Evaluate trade-offs between lightweight embedded storage and scalable client-server relational databases.

Action: I chose SQLite for default development because it is serverless, zero-config, file-based, and easy to run in unit tests. However, SQLite lacks true concurrent write capabilities due to database-level file locking. For production/docker deployments, I configured SQLAlchemy to switch seamlessly to MySQL (via DATABASE_URL environment variable), which supports fine-grained row-level locking, high concurrency, and replication.

Result: Developers can run pytest or uvicorn locally without installing database daemons, while production environments run robust MySQL containers with identical ORM models.

Category 2: Behavioral & Scenario-Based Questions

Q15: How does your application gracefully handle unexpected client disconnects, network drops, or server crashes?

Answer: Situation: Real-time apps frequently experience abrupt client disconnects (closing tabs, loss of Wi-Fi) or server restarts, leaving stale user states.

Task: Ensure user online status is accurately updated and socket resources are freed upon disconnect.

Action: In the WebSocket route, the receive loop is wrapped in a try...except WebSocketDisconnect block. When a disconnect triggers, manager.disconnect(user_id) immediately removes the socket from active_connections, updates is_online = False in the database, and broadcasts an updated online user list. Additionally, I added a FastAPI lifespan shutdown hook that bulk-resets all users' is_online = False on server restarts.

Result: Eliminates ghost online users and ensures clean connection teardown across abrupt failures.

Q16: If 10,000 users join a single popular chat room simultaneously, what bottlenecks will occur, and how would you optimize the system?

Answer: Situation: A high-concurrency surge (10,000 users in one room) creates exponential broadcast fan-out (1 message = 10,000 socket writes).

Task: Identify bottlenecks and propose scalable mitigation strategies.

Action: I identified three primary bottlenecks: CPU serialization overhead in JSON encoding, single-node network socket bandwidth, and database write throughput. To optimize: 1) Offload message persistence to an asynchronous background task queue (Celery/RabbitMQ or Redis Streams) so socket broadcasts aren't blocked by DB disk writes. 2) Implement message batching/throttling on high-frequency rooms. 3) Scale horizontally with Redis Pub/Sub across multiple ASGI worker nodes.

Result: Decouples message ingestion from delivery, scaling capacity linearly with worker nodes.

Q17: What security threats affect real-time WebSocket applications, and how did you mitigate Cross-Site Scripting (XSS) and message spam?

Answer: Situation: Chat applications are vulnerable to XSS injection (malicious script tags in chat messages) and spam flooding.

Task: Secure both API endpoints and WebSocket channels against abuse.

Action: 1) XSS Prevention: The vanilla JS frontend uses `textContent` and `innerText` when rendering chat message content rather than unescaped `innerHTML`, ensuring script tags execute as plain text string literals. 2) Authentication: All WS connections require JWT validation during handshake. 3) Spam Mitigation: Input validation is enforced via Pydantic schemas (min/max length limits), and frontend key-debounce prevents event flooding.

Result: Ensures client-side rendering safety and protects backend resources against malicious payloads.

Q18: Can you give an example of how you refactored code to improve maintainability and clean architecture principles?

Answer: Situation: Initial prototype code had WebSocket handling, database queries, and business logic tightly coupled inside a single `main.py` file.

Task: Refactor the codebase into a modular, clean software architecture.

Action: I restructured the application into clear layer responsibilities: `app/models/` (SQLAlchemy ORM models), `app/schemas/` (Pydantic validation), `app/services/` (Auth and business logic), `app/api/` (REST routes), and `app/websocket/manager.py` (WebSocket ConnectionManager broker). I also defined a unified JSON message protocol (`{ type, payload }`).

Result: Improved code readability, made components independently testable with `pytest`, and simplified adding new message types.

Q19: How would you approach adding a new feature, such as image/file attachments or read receipts, to this existing codebase?

Answer: Situation: Stakeholders request support for image attachments and read receipts.

Task: Plan and implement feature extensions cleanly without breaking existing message flows.

Action: 1) File Attachments: Create an HTTP POST `/api/upload` endpoint accepting multipart form data. Save files to AWS S3 (or local media directory) and return the CDN URL. Extend the Message schema with an optional `attachment_url` field. Update WebSocket JSON payload to include `attachment_url` so frontend renders `<img>` tags. 2) Read Receipts: Add `is_read` boolean and `read_at` timestamp to Message model, and emit a `message_read` WS event when recipient views the chat.

Result: Extends functionality seamlessly without altering core socket protocols.

Q20: Looking back, what lessons did you learn from this project, and what would you do differently if building it again today?

Answer: Situation: Reflecting on software architecture and design decisions made throughout development.

Task: Identify lessons learned and areas for future improvement.

Action: Key takeaways included the importance of decoupling socket lifecycles from DB connections early and standardizing WebSocket JSON schemas from day one. If rebuilding today: 1) Database Migrations: Integrate Alembic from the start for tracking schema revisions instead of relying on `Base.metadata.create_all()`. 2) Pub/Sub Broker: Implement Redis Pub/Sub out of the box to enable multi-node horizontal scaling. 3) Frontend Framework: Transition vanilla JS to React/TypeScript for cleaner component state management as UI complexity grows.

Result: Demonstrates continuous learning, architectural foresight, and production readiness.